

MBEYA UNIVERSITY OF SCIENCE AND TECHNOLOGY



GROUP ASSIGNMENT

DEPARTMENT: COMPUTER SCIENCE AND AND ENGINEERING

COURSE: COMPUTER SCIENCE (3RD YEAR).

**MODULE NAME: APPLICATION DEVELOPMENT FOR MOBILE
DEVICES**

MODULE CODE: CS 8306

FACILITATOR: SIR. AMANI RUKOIJO

GROUP MEMBERS:

SN	NAME	REG NO	CA
1.	GOODLUCK KENEDY	23100533350071	CA/BCS/25/7447
2.	GERALD BERNALD WILLISON	23100533350079	CA/BCS/25/13404
3.	GRACE A. MWAMBELO	23100533350066	CA/BCS/25/11926
4.	MIRIAM ELIAS MWANGA	23100533350080	CA/BCS/25/13401

Smart Attendance System Report

1. Introduction

1.1 Problem Description

Managing student attendance in educational institutions is traditionally a manual and paper-based process, which is highly inefficient. Teachers often spend valuable lecture time performing roll calls, and paper records are prone to errors and loss. Furthermore, tracking attendance trends over time for each student becomes difficult, making it challenging to identify absenteeism patterns or intervene in a timely manner.

1.2 Motivation

The motivation for developing the Smart Attendance System is to leverage technology to improve accuracy, efficiency, and reliability in attendance management. By integrating QR codes and a web-based platform, the system aims to automate the registration process, reduce human error, and provide administrators with real-time access to attendance records. Additionally, by making the system a Progressive Web Application (PWA), students and administrators can access it conveniently from any device without the need for a native app installation.

1.3 Project Objectives

- Develop a web-based system for automated attendance tracking using dynamic QR codes.
- Ensure the system is secure, preventing unauthorized access and fraudulent attendance marking.
- Provide a responsive, Progressive Web Application (PWA) for cross-device compatibility.
- Implement role-based access for students and administrators.
- Maintain a centralized database to facilitate easy retrieval, reporting, and analysis of attendance data.

- Reduce administrative workload and improve lecture efficiency.

2. System Requirements

2.1 Functional Requirements

- Students must be able to register for lectures with their registration number and personal details.
- Admins must be able to login, manage, and view attendance records.
- The system must generate dynamic QR codes for student registration.
- Attendance data should be automatically logged upon registration.
- The system must display real-time date and time for lectures and logging purposes.
- Alerts should be displayed upon successful registration or errors.
- Ability for students to edit registration details if allowed by rules.

2.2 Non-Functional Requirements

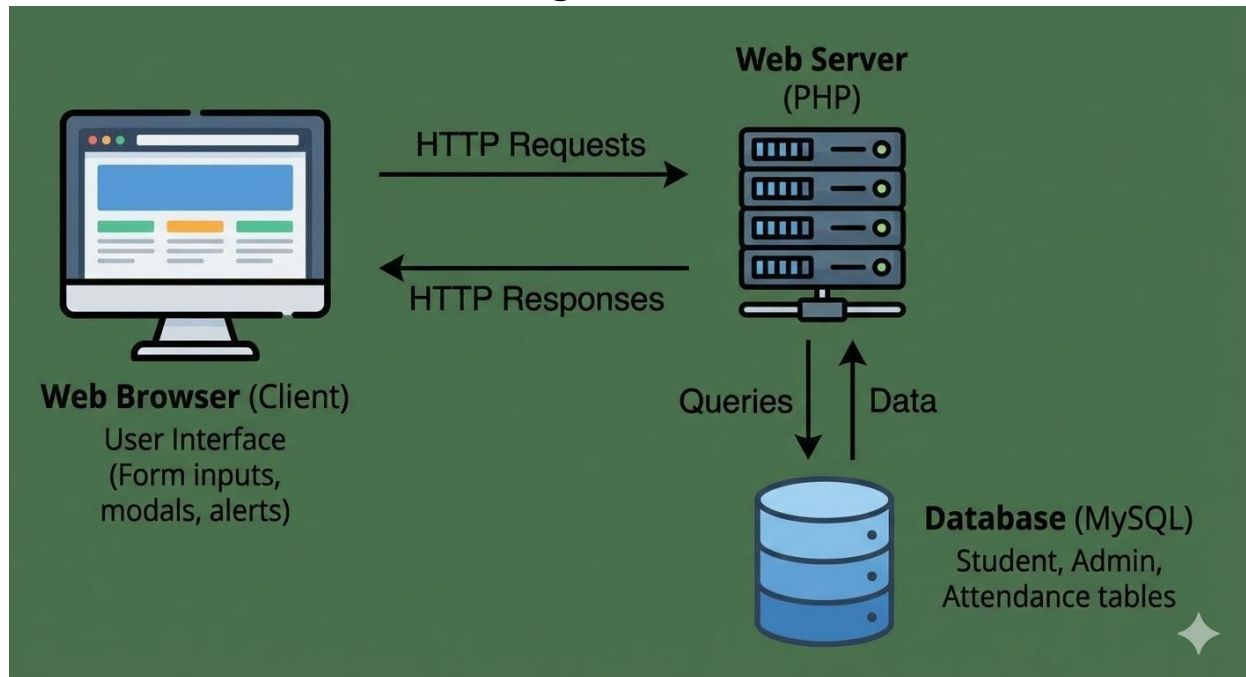
- **Security:** Ensure sensitive data is protected and unauthorized access is prevented.
- **Performance:** The system should load within three seconds and handle multiple concurrent users.
- **Usability:** Responsive design, user-friendly forms, and accessible interface.
- **Reliability:** The system must remain stable under high usage.
- **Maintainability:** Code should be modular, well-documented, and easy to update.
- **Scalability:** Database and server should support growth in the number of users and records.

3. System Architecture

3.1 Overall System Design

The Smart Attendance System is designed using a client-server architecture. The client side includes students and administrators accessing the web interface through browsers. The server side, powered by PHP, processes requests, handles database operations, and returns responses. MySQL serves as the database, storing all student information, attendance records, and administrator credentials. The architecture is designed for scalability, security, and efficiency.

3.2 Client-Server Architecture Diagram



This diagram illustrates the communication flow between the client (user browser) and the server, highlighting how data moves between the interface and database.

3.3 Database Schema (ERD)

- **Tables:**
 - students (id, fullname, course, year, reg_no, created_at)
 - attendance (id, student_id, date, status)
 - admins (id, username, password, role)
- **Relationships:**

students (1) -----< attendance (many)

admins (1) manage attendance (many)

The ERD ensures normalization and reduces data redundancy. Foreign keys enforce relationships between students and attendance records.

3.4 Authentication Mechanism

- Admins login with username and password.
- Passwords are hashed using bcrypt for security.
- PHP sessions manage user login status, ensuring only authorized access to restricted areas.
- Student registration is open but validated against registration number patterns.

3.5 Security Measures Used

- HTTPS for all communication.
- Input validation and sanitization to prevent SQL injection and XSS attacks.
- Role-based access control separating student and admin functionalities.
- Service workers in PWA caching only non-sensitive resources.
- Passwords are hashed and never stored in plaintext.

4. Implementation Details.

4.1 Technologies Used.

- **Frontend:** HTML5, CSS3, JavaScript, Bootstrap 5
- **Backend:** PHP 8.x
- **Database:** MySQL 8

- **Other:** AJAX, JSON responses, Progressive Web Application (manifest, service worker)

4.2 Frontend Implementation

- Card-based responsive layout with gradient backgrounds.
- Modal forms for student registration and admin login.
- Real-time clock display on navbar.
- Form validations using HTML5 and JavaScript.
- Alerts for successful registration disappear after 2 seconds.
- PWA features enabled for mobile and desktop users.

4.3 Backend Implementation

- PHP scripts handle student registration, admin login, and attendance record management.
- AJAX used to send form data asynchronously and receive JSON responses.
- PHP sessions ensure secure user login tracking.
- Error handling implemented for database operations.

4.4 Database Design.

- Normalized tables to avoid redundancy.
- Indexed reg_no for fast lookups.
- Foreign keys enforce relationships.
- Designed for scalability as student numbers grow.

4.5 Challenges Faced.

- Ensuring unique registration numbers while allowing students to edit if rules permit.
- Displaying real-time alerts without page reloads.

- Implementing PWA functionality without compromising security.
- Making the system responsive and compatible across devices.

4.6 Assumptions Made

- Registration numbers start with '231'.
- Admin accounts are pre-created and managed securely.
- Users have modern browsers supporting PWA features.

5. Security Considerations

5.1 Authentication.

- Admins must provide valid credentials.
- Student registration is limited to validated registration numbers.
- Sessions expire after inactivity.

5.2 Password Storage.

- Passwords hashed with bcrypt.
- No plaintext passwords are stored.

5.3 Data Protection

- HTTPS encrypts data in transit.
- Input sanitization prevents SQL injection and XSS.
- Role-based access control ensures restricted access.

5.4 Access Control Implementation

- Students limited to registration pages.
- Admins have full access to manage attendance logs.
- Sensitive pages check session and role before granting access.

6. Testing & Results

6.1 Screenshots

Placeholder for screenshots: homepage, registration modal, admin login, dashboard, PWA install prompt.

6.2 Major Features Working

- Student registration form with validation and success alert.
- Admin login and dashboard access.
- Real-time clock display.
- PWA installable and offline-friendly UI.
- Alerts and feedback mechanism functioning as expected.

6.3 Incomplete Features

- Offline database operations.
- QR code scanning for automated attendance (planned for future release).

7. Future Work

7.1 Possible Improvements

- Integrate QR code scanning for automatic attendance.
- Admin dashboard analytics for attendance trends.
- Push notifications for students about missed lectures.
- Multi-language support for broader accessibility.

7.2 Scalability Considerations

- Move database to cloud infrastructure for high availability.
- Implement caching for high-traffic scenarios.
- Use RESTful API to separate frontend and backend for easier scalability.

Github repo link

[fonex6/student-attendance-system](https://github.com/fonex6/student-attendance-system)

Video Link

<https://youtu.be/qRTXvML5eiY>